This version of the contribution has been accepted for publication, after peer review (when applicable) but is not the Version of Record and does not reflect post-acceptance improvements, or any corrections. The Version of Record is available online at: http://dx.doi.org/10.1007/978-3-032-39892-5_35. Use of this Accepted Version is subject to the publisher's Accepted Manuscript terms of use
https://www.springernature.com/gp/open-research/policies/accepted-manuscript-terms

Agentic Text-to-SQL with Table-Level Retrieval for Protective Relay Settings

Gustavo L. B. Pereira[1,2], João Pedro G. Fernandes[1,2], João Pedro A. F. Matos[1,2], Vinícius A. Aguiar[1,2], Sávio S. T. de Oliveira[1,2], and Anderson S. Soares[1,2]

[1] Institute of Informatics, Federal University of Goiás, Goiânia, Brazil
{gustavobueno, joaopedro234567, vinicius_aguiar2}@discente.ufg.br
jpaguiar39@gmail.com, savioteles@ufg.br, anderson@inf.ufg.br
[2] Center of Excellence in Artificial Intelligence (CEIA), Brazil

Abstract. Protective relay configuration data is critical for power system operations, yet it remains locked in heterogeneous PDF manuals from multiple manufacturers with non-standardized formats and low-semantic parameter names. We propose an agentic Text-to-SQL system for natural-language access to relay configuration parameters, built on a structured relational database extracted from PDF tables and equipped with a ReAct-style agent that combines semantic retrieval with SQL execution. We systematically compare two retrieval strategies: Section-Level Retrieval (SLR), which indexes full document sections, and Table-Level Retrieval (TLR), which indexes per-table LLM-generated descriptions. On a domain-specific evaluation set of 112 questions over relay manuals from seven manufacturers, TLR achieves mean factual correctness of $[0.87; 0.91]$, outperforming SLR with the same backbone at lower token cost and latency, with the advantage consistent across both backbone models evaluated. The results indicate that retrieval granularity and backbone capability each contribute separately to accuracy and efficiency, with the strongest configuration combining table-level retrieval and the more capable backbone.

Keywords: Text-to-SQL · LLM Agents · Table Retrieval · Protective Relay Settings · Power Systems · Semantic Retrieval

1 Introduction

Protective relays play a central role in power-system operation by detecting fault conditions and triggering the actions required to isolate them. To operate correctly, each relay must be configured with parameters such as reach settings, pickup thresholds, and operating delays, which are periodically checked by protection and operations teams. In practice, however, these values are documented in manufacturer-issued PDF manuals that vary in structure, terminology, and table organization. Answering a question such as "What are the distance protection settings configured for the relay associated with line segment T-10?" may require

opening multiple documents, locating the relevant section, interpreting the appropriate table, and manually transcribing the requested values. This process is time-consuming and error-prone.

This paper addresses the problem of providing natural-language access to relay configuration data stored in such manuals. Given a user question and a corpus of heterogeneous PDF manuals, the goal is to recover the requested parameter values from the relevant extracted tables and return them together with their source document and page. This task is challenging because the relevant information may be distributed across many tables and documents, the correspondence between user wording and the underlying schema is often indirect, and returned values must remain traceable to their source for operational checking. The problem therefore involves not only retrieval, but also schema identification, user-intent disambiguation, and structured value access.

Existing approaches address only part of this setting. Text-to-SQL benchmarks [17,7] assume human-curated schemas with stable structures and meaningful column names, whereas our database is extracted from heterogeneous PDF manuals and reflects manufacturer-specific terminology. Retrieval-Augmented Generation (RAG) [6] is a natural practical baseline, but raw document chunking does not exploit table structure or the sparse, evolving schema that the configuration data induces, and does not support SQL-based access to individual parameter values; we include it as an external point of comparison rather than as the primary architecture. Prior natural-language interfaces for power systems [15,14] and table-oriented retrieval methods [12] have not, to our knowledge, been studied in the setting of manufacturer-issued relay manuals with sparse and evolving schemas.

To address this problem, we propose an agentic system for natural-language access to relay configuration parameters. At a high level, the system converts relay manuals into a sparse relational representation and answers user questions by combining semantic retrieval, schema inspection, and SQL execution. This design allows each returned value to remain linked to the underlying document and page. Within this architecture, we also examine one design choice with direct impact on system behavior: the granularity of the retrieval step. We compare *Section-Level Retrieval* (SLR), which retrieves larger document sections, and *Table-Level Retrieval* (TLR), which retrieves individual tables through semantic descriptions derived from the extracted content. In critical power-system operations, a correct answer without exact traceability to a specific table row is operationally unauditable. We therefore include it as an external point of comparison to demonstrate that our system matches standard RAG correctness while enforcing strict structural provenance.

The main contributions of this work are threefold. First, we introduce an extraction pipeline that converts heterogeneous relay-configuration PDFs into a provenance-aware sparse relational representation, preserving structural context such as section hierarchies, table titles, and page information. Second, we propose an agentic query architecture for relay-parameter access that combines semantic retrieval, schema inspection, and constrained SQL execution, alongside

a controlled comparison between section-level and table-level retrieval strategies under two agent backbone models. Finally, we present an experimental evaluation on a domain-specific set of 112 questions over relay manuals, supported by a structured evaluation protocol, a comparison against a chunk-based RAG baseline under the same backbone, and a detailed error taxonomy of the main failure modes observed in this setting.

2 Related Work

Text-to-SQL has grown steadily more capable, yet most recent systems assume relatively stable schemas with semantically meaningful column names. Spider [17] and BIRD [7] established cross-domain benchmarks, the latter with dirty values and external-knowledge requirements. Recent methods attack schema understanding through explicit prompt decomposition (DIN-SQL [10]), interactive query execution with a single agent (RAISE [3]), multi-agent collaboration (MAC-SQL [13]), and constrained decoding (PICARD [11]). All of them assume a legible, stable schema, an assumption that fails when column identifiers are opaque mnemonics and the column set itself grows as documents are ingested.

Work on table retrieval emphasises that structured content must be represented with its local context. Burdick et al. [2] distinguish table *extraction* from table *understanding* and locate most application value in the latter. Herzig et al. [4] show that dense retrievers designed for table context outperform generic text retrievers on open-domain table QA by narrowing the lexical gap to the tabular content. TableRAG [12] generates textual representations of tables, indexes them for retrieval, and uses a language model to produce SQL or Python for execution on web-table benchmarks.

Agentic approaches to structured data build on ReAct [16], which interleaves reasoning with tool calls. TAG [1] argues that Text-to-SQL alone is insufficient for realistic queries and proposes augmenting generation with tool use over structured data; RAISE and related systems [3] instantiate this by iterating between query execution and hypothesis refinement. Natural-language interfaces for power systems, in turn, have targeted operational tasks: SCADA-NLI [15], for example, combines intent classification with NL-to-SQL over a curated schema for distributed SCADA control, rather than over configuration data recovered from vendor manuals.

These agentic pipelines address adjacent but distinct problems, emphasizing schema compression, exploration-cost control, or orchestration and guardrails for enterprise deployments. Our agent loop reuses the reason-execute-observe pattern shared by these systems, but the contribution lies elsewhere: we target the retrieval layer that feeds the agent, on a sparse, dynamically generated schema populated with opaque vendor mnemonics, with content retrievable only at the granularity of individual tables, and with answers that must remain traceable to specific rows and source pages of Portuguese manufacturer manuals. To our knowledge, no prior system simultaneously satisfies these conditions.

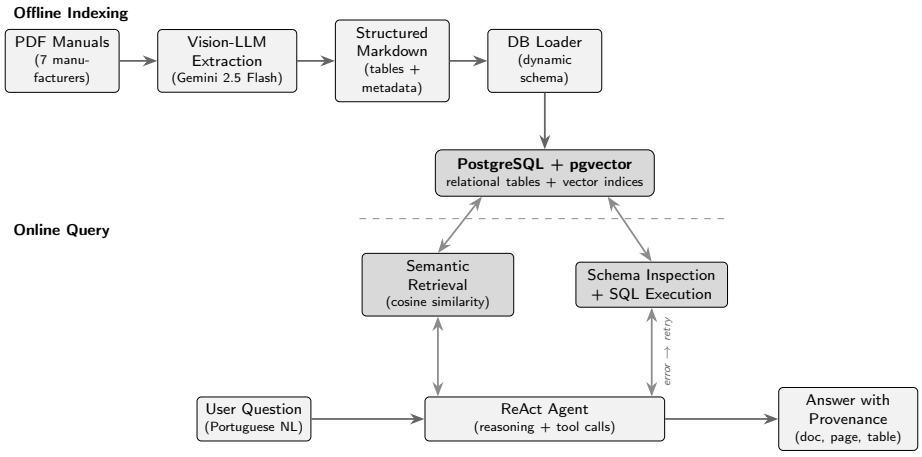


Fig. 1. Relay manuals are parsed into a sparse relational database together with vector indices. At query time, the retrieval layer, in one of its two variants (SLR or TLR), identifies the tables relevant to the user's question and passes them to a ReAct-style agent, which generates and executes SQL and returns an answer linked to the originating document and page.

3 Proposed Approach

3.1 Problem Formulation

We consider a corpus $\mathcal{D}$ of manufacturer-issued PDF manuals documenting the protection settings of installed relays. The corpus is heterogeneous along every axis that matters for retrieval: each manual follows its own section structure, uses its own table layouts, and labels parameters with vendor mnemonics such as `Z1RANG` or `En_IOElim` whose meaning is not recoverable from the identifier alone.

Given $\mathcal{D}$ and a natural-language question q in Portuguese, the task is to produce an answer a that jointly satisfies three conditions. i) *Correctness*: every value in a must match the value recorded in the corresponding table of some manual in $\mathcal{D}$; ii) *Completeness*: a must cover every parameter requested by q, including requests whose answer spans multiple tables or multiple manuals; and iii) *Provenance*: every value in a must be linked back to a specific row of a specific table, identified by document and page, so that an operator can verify it against the source manual.

The defining difficulty imposed by this setting is a *semantic gap* between q and the schema induced by $\mathcal{D}$: an engineer may ask about "the distance protection zone 1" while the corresponding parameter appears in the underlying tables as a column named `Z1RANG`, and neither lexical overlap nor generic semantic similarity over the raw identifiers reliably connects one to the other. This gap, together with the three conditions above, shapes the design choices discussed in the remainder of this section and summarised in Figure 1.

3.2 Data Extraction and Indexing

As shown in the Offline Indexing row of Figure 1, each PDF manual is first converted into a structured intermediate representation by a generative vision-language model (Gemini 2.5 Flash, chosen for its favourable balance of extraction quality, throughput, and cost relative to larger alternatives), which outputs Markdown preserving section headings, table titles, page numbers, and the row-column layout of every detected table. The Markdown tables are then parsed into database rows by the DB Loader and persisted into the schema described below.

Because a vision model cannot be trusted as cell-level ground truth, we do not attempt to guarantee extraction correctness at ingestion time. Instead, every stored row carries provenance fields (*document*, *section*, *table title*, *page*), so any returned value can be traced back to a specific source page for human verification.

These rows populate the PostgreSQL + pgvector store of Figure 1, organised as four tables that separate value storage from schema inspection and semantic retrieval. `parameters` holds the actual relay configuration values and is the target of every SQL query. The remaining three tables guide retrieval: `section_tables_schema` exposes column names and sample rows for schema inspection before SQL; `sections` and `table_descriptions` carry vector indices consumed by SLR (Section 3.3) and TLR (Section 3.4) respectively.

The layout of `parameters` itself is the one non-standard element of this schema. Rather than a fixed set of columns agreed upon in advance, each unique parameter name encountered during ingestion becomes its own column `[param_col`$_1$`]`, `...`, `[param_col`$_k$`]`, created on demand so that the set of columns can grow monotonically as new manuals are ingested. Any given row populates only the columns relevant to its source table and leaves the rest `NULL`. This *dynamic wide-sparse* layout accommodates the heterogeneity of vendor-specific parameter vocabularies without requiring a fixed parameter ontology or manual schema redesign. We adopted this layout over alternatives such as entity-attribute-value or JSONB-typed columns for reasons of simplicity: keeping each parameter as a first-class column lets the agent reuse standard `SELECT [param_col] FROM parameters` templates and the same schema-inspection step across every query, rather than reasoning about nested keys or self-joins. The trade-off, discussed in Section 5, is that the column space grows monotonically with the corpus and that NULL density is high.

On the corpus used in this work, `parameters` materialises into 100,206 rows across 249 dynamic parameter columns, of which only 1.48% of cells are non-null; the median row populates just 3 parameter columns. This density regime is what the retrieval layer of Sections 3.3 and 3.4 is designed to exploit: any given question touches a small subset of the sparse space, and identifying that subset before issuing SQL is more tractable than scanning the full table.

Both `sections.embedding` and `table_descriptions.embedding` are 768-dimensional vector columns produced by the `gemini-embedding-001` model and stored in Postgres with the pgvector extension (IVFFlat index). What differs between SLR and TLR is the *unit* that is embedded, not the embedding infrastructure.

3.3 Section-Level Retrieval (SLR)

Within the Online Query flow of Figure 1, SLR instantiates the Semantic Retrieval tool at the granularity of whole document *sections*. A section is a natural first choice because manuals are authored section by section and tables discussing the same protection function typically share a heading; however, the raw text of such sections is dominated by dense tables and vendor mnemonics, offering little natural-language surface for an embedder. We therefore embed not the section body but a short metadata template concatenating the section's identifying attributes (*"Parameters of relay [equipment_code], manufacturer [manufacturer], for [asset_type], function [ansi_code] ('[ansi_description]'), referring to section [section_name]."*), stored in `sections` together with its embedding.

At query time, the agent uses four sequential tools:

1. `list_equipment(asset_type, manufacturer, equipment_code)` (entity-grounding step preceding the blocks of Figure 1): resolves underspecified manufacturer or model identifiers, anchoring the query to a concrete relay record.
2. `get_similar_sections(query, asset_type, manufacturer, equipment_code)` (Semantic Retrieval block of Figure 1): performs a cosine-similarity search over `sections.embedding`, optionally filtered by any combination of the fields resolved in the previous step, and returns the top-k matching sections ($k = 20$ for SLR and $k = 10$ for TLR, with the larger pool in SLR compensating for the coarser retrieval unit).
3. `get_tables_schema_from_section(section_id)` (Schema Inspection portion of the Schema Inspection + SQL Execution block): returns the list of tables inside a retrieved section, including column names and sample rows, so that the agent can inspect the schema before writing SQL.
4. `execute_sql_query_engop(sql)` (SQL Execution portion of the same block): executes the generated query against `parameters` and returns the results.

The fields returned by `list_equipment` act as hard filters on the cosine-similarity search performed by `get_similar_sections`, restricting it to the slice of the index that belongs to the resolved relay; without this grounding, the query would compete against embeddings from every manufacturer and asset type in the corpus.

Because the embedding key summarises metadata shared by every table under a section, sibling tables map to the same retrieval vector: a distance-protection section with a dozen parameter tables is represented by a single embedding, and the agent must call `get_tables_schema_from_section` to disambiguate before committing to SQL. The minimum tool-call sequence in SLR is therefore four steps.

3.4 Table-Level Retrieval (TLR)

Keeping the Online Query flow of Figure 1 fixed, the second architecture, TLR, reuses the same Semantic Retrieval tool but shifts its retrieval unit from sections

to individual *tables*. For each table extracted from $\mathcal{D}$, a language model generates a natural-language description that summarizes the table's content and its place in the surrounding document. The prompt exposes the document metadata (manufacturer, asset type, equipment code, ANSI protection function, section name, table title) and a sample of rows, and instructs the model to return a short paragraph in Portuguese stating what the table contains and when it would be consulted. Each description is stored in the `table_descriptions` table, one row per extracted table, alongside its vector embedding. Embedding LLM-generated table descriptions also appears in TableRAG [12] and in open-domain table retrieval [4]; TLR differs in its integration with a sparse-schema, provenance-aware relational backend, the Portuguese vendor-mnemonic setting with a dynamically growing column space, and the controlled contrast against SLR holding embedding infrastructure constant.

The unit embedded by TLR is therefore the description paragraph rather than the table's raw content or a section-level template, with the embedding model, dimensionality, and index unchanged from Section 3.2.

The tool-call sequence in TLR has three steps:

1. `list_equipment(...)` (entity-grounding step preceding the blocks of Figure 1): entity-level grounding as in SLR, whose output restricts the subsequent semantic search to the resolved relay.
2. `get_similar_tables(query, asset_type, manufacturer, equipment_code)` (Semantic Retrieval block of Figure 1, with the schema-inspection portion of the next block returned inline): performs a cosine-similarity search over `table_descriptions.embedding`, optionally filtered by the same fields as `get_similar_sections`, and returns the top-k matching tables together with their `column_names` and `sample_rows` inline.
3. `execute_sql_query_engop(sql)` (SQL Execution portion of the Schema Inspection + SQL Execution block): SQL execution against `parameters`.

The retrieval key corresponds to one table rather than to the metadata shared by all tables in a section, so the section-level collapse does not arise. The schema information that SLR obtains through a separate call is returned inline by `get_similar_tables`, shortening the minimum tool-call sequence from four steps to three. Which arrangement resolves a given query more reliably, and at what cost, is an empirical question taken up in Section 4.3.

3.5 Agent Design

Both architectures share the ReAct Agent block of Figure 1, built on top of the Google Agent Development Kit using a ReAct-style [16] controller that interleaves reasoning traces with tool calls and updates its plan after every observation. The only component that varies across experiments is the backbone model that produces those traces; we evaluate Gemini 3 Flash Preview and Gemini 2.5 Flash, with sampling hyperparameters reported in Section 4.3.

The system prompt encodes three invariants derived from iterative error analysis of earlier runs of the system (Section 4.4); each targets a specific failure mode that the tool interface alone did not rule out. *Mandatory entity grounding:* the agent must call `list_equipment` before any retrieval call, so that semantic retrieval runs against the slice of the index that belongs to the resolved relay rather than against the full corpus. *Schema-before-SQL:* the agent must inspect column names and sample rows before emitting any SQL, preventing queries that reference non-existent identifiers. *Completeness check:* before returning a final answer, the agent must verify that every parameter requested by the user is accounted for, re-querying when necessary.

SQL execution itself doubles as an implicit verifier: a query with an invalid column name or a failed cast returns the database error to the agent, which must reformulate before proceeding (the "error $\rightarrow$ retry" arc in Figure 1). This execution-guided refinement loop mirrors the interactive schema exploration pattern of recent agentic Text-to-SQL systems [3], and is essential for navigating the sparse, low-semantic schema introduced in Section 3.2.

4 Experiments

4.1 Evaluation Set Construction

Public Text-to-SQL benchmarks such as Spider [17] and BIRD [7] assume curated schemas with semantically meaningful column names and do not target relay configuration manuals. For lack of a public alternative, we build an internal evaluation set from the same corpus $\mathcal{D}$ that the indexing pipeline consumes, using a three-stage automated pipeline. A classifier first filters out pages that lack sufficient content (headings, glossaries, cross-reference indices). A generator agent (Gemini 2.5 Flash) then produces, for each remaining page, a Brazilian-Portuguese question, a reference answer, and a set of verbatim snippets supporting the answer; questions are required to be self-contained, realistic, and contextualised, and may span one or several pages of the same document. A validator agent finally checks each triple against seven criteria (self-containment, realism, use of domain context, specificity, grounding to the snippets, table-level precision, and verbatim sufficiency) and returns structured feedback for up to three regeneration attempts before the page is discarded.

The pipeline produced **112 questions** from 46 manuals covering transmission-line and power-transformer assets across seven manufacturers (Siemens, GE, ABB, Alstom, AREVA, NR, SEL), and all 112 form the evaluation set used in Section 4.3.

4.2 Evaluation Methodology

Factual Correctness Metric. Standard metrics such as exact match and F1 are unsuitable for our setting: relay configuration answers may include numeric values, parameter lists, or mixed tables, and extracting a canonical form would introduce

additional error sources [19]. Following the LLM-as-judge paradigm [18], we score each response with G-Eval [8], using Gemini 3 Flash Preview as the primary judge under the criterion: *"Score 1.0 if all factual information from the reference is present and correct in the generated response; 0.5 if the response contains part of the required information; 0.0 if the response is completely different or incorrect."* The three-level scale $\{0.0, 0.5, 1.0\}$ separates completely incorrect from partially correct answers, capturing a common failure mode in which the agent retrieves the right table but returns only some of the requested parameters.

Judge Reliability. Because one of the evaluated backbones shares its model family with the primary judge, a single-judge protocol would be open to the objection of self-preference bias [9]. We therefore re-score every (question, response) pair with GPT-5.4, a judge from a different provider, under the same criterion. Agreement is substantial to near-perfect on the scale of Landis and Koch [5], with Cohen's κ between 0.73 and 0.84 across configurations and no 0.0-vs.-1.0 disagreements among the 448 evaluated pairs. GPT-5.4 is systematically stricter, with most discrepancies at the 1.0/0.5 boundary, and the ranking of architectures and backbones is preserved under both judges. We report factual correctness as the interval [GPT-5.4; Gemini] throughout, reading the lower bound as the conservative cross-family estimate.

Additional Metrics. We also report wall-clock latency, tool-call count, prompt token consumption, and API cost, which are relevant for production deployment in utility environments. Each (question, configuration) pair is evaluated in a single run; standard deviations in Section 4.3 are over the 112 questions, not over repeated runs of the same question.

4.3 Main Results

Table 1 reports the 2×2 grid crossing both retrieval architectures with both agent backbone models, together with a chunk-based RAG baseline evaluated under Gemini 3 Flash Preview, on the full 112-question evaluation set. All runs share the system prompt described in Section 3.5. Gemini 3 Flash Preview is sampled at temperature 1.0, following the default setting recommended in the official model documentation[3]; Gemini 2.5 Flash uses temperature 0.2 to reduce variance on the more error-prone backbone.

Across both backbones, TLR yields higher factual correctness and lower latency, tool-call count, prompt-token count, and cost than SLR (Table 1). The largest separation is on efficiency: SLR with Gemini 3 Flash Preview averages 13.37 tool calls and 98,437 prompt tokens per query (2.7× the TLR count), leading to a total cost of $5.95 against $2.33 for TLR. SLR's mean latency is pulled up by a single outlier of 324.4 s in which the agent entered a reformulation loop over a low-semantic identifier; the median latency for the same configuration

[3] `https://ai.google.dev/gemini-api/docs/gemini-3#temperature`, accessed 2026-04-24.

Table 1. Results on 112 questions for both retrieval architectures (SLR, TLR) under two agent backbone models, and a chunk-based RAG baseline under Gemini 3 Flash Preview. Factual correctness is reported as [GPT-5.4; Gemini] interval; counts and standard deviation use the primary Gemini judge. Best result per row in **bold**.

	Gemini 3 Flash Preview			Gemini 2.5 Flash	
Metric	**SLR**	**TLR**	**RAG**	**SLR**	**TLR**
Factual correctness (112 questions)					
Mean [GPT-5.4; Gemini]	[0.75; 0.80]	**[0.87; 0.91]**	[0.81; 0.88]	[0.28; 0.30]	[0.58; 0.62]
Score = 1.0 (perfect)	85	**96**	90	22	57
Score = 0.5 (partial)	10	12	16	24	25
Score = 0.0 (wrong)	17	**4**	6	66	30
Std. deviation	0.37	**0.23**	0.27	0.40	0.43
Response latency					
Mean (s)	69.7	45.8	60.6	49.2	**43.6**
Std. dev. (s)	42.9	17.5	20.2	14.4	**10.4**
Maximum (s)	324.4	140.8	150.6	104.6	**72.7**
Efficiency					
Mean tool calls	13.37	3.56	**1.25**	4.62	3.25
Maximum tool calls	94	14	**4**	10	6
Mean prompt tokens	98,437	36,483	**25,506**	33,811	30,011
Cost (112 queries)					
Total cost (USD)	$5.95	$2.33	$1.63	$1.30	**$1.22**

is 59.9 s. The token gap follows directly from the section-level collision described in Section 3.3: when the first retrieval attempt fails to pinpoint the right table, the agent re-queries and accumulates context across attempts.

The backbone effect is large under both architectures: under TLR, mean factual correctness rises from [0.58; 0.62] with Gemini 2.5 Flash to [0.87; 0.91] with Gemini 3 Flash Preview. Manual inspection suggests that Gemini 3 Flash Preview more often proceeds through the tool sequence without pausing to request user clarification, which removes a share of the dominant error class in Section 4.4.

Combining factual correctness with cost yields $0.024 per correct answer for TLR+Gemini 3 Flash Preview against $0.070 for SLR with the same backbone, a 2.9× improvement that follows from more precise first-attempt retrieval. Within the 2×2 grid, TLR with Gemini 3 Flash Preview is not dominated on any metric: configurations with lower latency or cost score below 0.65 in mean factual correctness, and no configuration scores higher on correctness. We refer to this point as the *best configuration* in the remainder of the paper.

Comparison against a chunk-based RAG baseline. We additionally evaluate a chunk-based RAG baseline on the same 112-question set. It reuses the Gemini-based extraction pipeline and the `gemini-embedding-001` index, but replaces

the relational backend and the SQL step with a single retrieval tool over recursive text chunks ($k = 10$), with no schema inspection or SQL execution; it runs under Gemini 3 Flash Preview to isolate architecture from backbone. Relative to the baseline, TLR with the same backbone shows a point-estimate advantage of 0.03 to 0.06 in mean factual correctness ($[0.87; 0.91]$ against $[0.81; 0.88]$) that does not reach significance (see below), reduces mean latency from 60.6 s to 45.8 s, and incurs higher prompt-token usage (36,483 against 25,506 on average) and cost ($2.33 against $1.63). The structured backend therefore delivers on latency and on architectural properties not captured by this metric, namely native row-level provenance and a query interface aligned with completeness and auditability requirements, while the baseline remains a competitive anchor on correctness, tokens, and cost.

Significance of pairwise contrasts. We assess pairwise differences with a paired bootstrap over the 112 questions, Holm-correcting across six contrasts. The four contrasts within the 2×2 grid are all significant under both judges: the tightest one (TLR vs. SLR under Gemini 3 Flash Preview) yields $\Delta = +0.107$ $[+0.031, +0.183]$ (Gemini, $p_{\text{Holm}} = 0.028$) and $\Delta = +0.121$ $[+0.040, +0.205]$ (GPT-5.4, $p_{\text{Holm}} = 0.018$); the remaining three show $\Delta \geq 0.29$ with $p_{\text{Holm}} \leq 0.001$. Per-contrast deltas agree within 0.02 between judges, so the ranking does not depend on the judge. The backbone gap is descriptively narrower within TLR ($\Delta \approx +0.29$) than within SLR ($\Delta \in [+0.47, +0.50]$), consistent with more precise retrieval absorbing part of the loss from a weaker backbone. Neither contrast against the baseline reaches significance (TLR with Gemini 3 vs. RAG: $\Delta = +0.036$ and $+0.058$, $p_{\text{Holm}} = 0.33$ and 0.28; SLR with Gemini 3 vs. RAG also fails to cross zero): at $n = 112$, the structured backend's advantage over the chunk-based baseline is supported on latency, not on correctness. Thus, the primary advantage of the structured agentic backend over standard chunk-based RAG lies not in raw correctness, but in its ability to enforce native row-level provenance and exact schema adherence, while simultaneously delivering a 24% reduction in mean latency.

4.4 Error Analysis

We manually inspected the incorrect and partial responses across the four grid configurations and grouped the recurring failures into six classes (Table 2), comparing the weakest configuration (SLR with Gemini 2.5 Flash) against the best one (TLR with Gemini 3 Flash Preview). The discussion below attributes each class to the design lever that most directly addresses its root cause.

The six classes split into three groups according to the design lever that addresses them.

Addressed by TLR descriptions (E1, E5). E1 is the most frequent error class in the weakest configuration: parameter names such as `Z1RANG` or IEC 61850 composites like `B01.Dur_Recv_Long_GOOSE` are syntactically constructed strings with no meaningful representation in general-purpose embedding models, causing the retrieval step to return unrelated sections. Embedding LLM-generated table

Table 2. Error taxonomy derived from manual inspection of incorrect and partial responses across all four configurations. Counts are affected questions in each endpoint of the experimental grid (Gemini judge).

		Count		
Type	**Name**	**Weakest**	**Best**	**Root Cause**
E1	Retrieval Miss	33	0	Vendor-specific names (*e.g.*, IEC 61850 composites) absent from embedding vocabulary.
E2	Entity Confusion	9	5	Agent selects correct table but wrong indexed row (*e.g.*, wrong winding or output terminal).
E3	Incomplete Response	21	5	Multi-value queries satisfied after first partial match.
E4	Logic Traversal	10	2	FlexLogic/TIA Portal programs require dependency-graph traversal.
E5	Term Ambiguity	8	1	Mismatch between natural-language terms and vendor mnemonics.
E6	Multi-row Aggregation	9	3	Range queries (*e.g.*, items 17 to 26) return partial lists.
	Total	*90*	*16*	

descriptions instead of raw identifiers closes this gap entirely on the evaluation set: E1 drops from 33 affected questions to 0 in the best configuration, including the compound IEC 61850 cases that we expected to remain hard. E5 follows the same pattern at the level of natural-language wording: when an engineer uses domain terminology that does not match vendor-specific column names, the description text restates each table's content in natural language, reducing the mismatch from 8 affected questions to 1.

Addressed by the agent prompt (E3, E6). E3 and E6 stem from premature termination on multi-value or range queries, where the agent stops after the first partial match instead of completing the requested set. The completeness-check rule of Section 3.5 reduces both classes (E3 from 21 to 5, E6 from 9 to 3), and the more precise first-attempt retrieval of TLR amplifies the effect by leaving fewer queries in a state where the agent must reformulate. Neither lever eliminates the classes: queries over wide index ranges and over heterogeneous parameter sets remain the hardest cases and would benefit from explicit range-filter indexing.

Reduced indirectly, with residual hard cases (E2, E4). E2 and E4 are not direct targets of either retrieval-side or prompt-side changes, yet both decrease across the grid: E4 from 10 to 2 and E2 from 9 to 5. The drop in E4 is plausibly attributable to a combination of the more capable backbone, which handles

dependency-graph reasoning over GE FlexLogic equations and Siemens TIA Portal connection matrices more reliably, and the more focused context that TLR supplies, which leaves the model with fewer irrelevant tables to traverse. E2 is reduced the least and accounts for the four zero-score responses that persist in the best configuration (a fifth E2 case scores 0.5): the agent retrieves the correct table but returns data from the wrong sub-object, such as winding 2 values when winding 3 was requested, or the wrong output terminal from a dense assignment table. This failure is not resolvable by retrieval granularity or by prompt rules and would require cell-level row identification.

5 Conclusion

We presented an agentic Text-to-SQL system for natural-language access to relay configuration parameters in heterogeneous PDF manuals, and evaluated two retrieval architectures and two backbones on 112 questions. TLR with Gemini 3 Flash Preview is the best configuration within the 2×2 SLR/TLR grid, not dominated on any metric, raising mean factual correctness from $[0.75; 0.80]$ to $[0.87; 0.91]$ over SLR with the same backbone while also lowering token cost and latency.

Against a chunk-based RAG baseline under the same backbone, this configuration is not statistically distinguishable in factual correctness at $n = 112$; its advantage lies instead in lower mean latency and in native row-level provenance and schema adherence that the chunk baseline does not provide.

The error analysis indicates that the table-description index essentially eliminates retrieval-miss and term-ambiguity failures, leaving row-level entity confusion within correctly retrieved tables as the dominant residual failure mode. Our results demonstrate that the proposed architecture matches the empirical performance of unstructured baselines while satisfying the strict provenance and auditability requirements of electrical engineering workflows.

Limitations. Each (question, configuration) pair is evaluated in a single run, both backbone models belong to the same provider's family, and factual correctness is measured by an LLM-as-judge protocol. The external comparison is restricted to a simple chunk-based RAG baseline; stronger hybrid retrieval baselines are not covered, and the sample size of 112 limits the statistical power of the external comparison. The evaluation set itself is built from synthetically generated questions rather than from queries collected in operational practice. Finally, ingestion relies on a vision-language model to extract tables from PDF manuals, and we report no quantitative cell-level fidelity audit; provenance fields make every value traceable to a source page for human verification, but they do not quantify the ingestion-error rate that may propagate to retrieval and to the correctness scores reported here.

Future Work. Two extensions stand out. First, replacing the synthetic evaluation set with queries collected from practising protection engineers, balanced across

manufacturers and asset classes, would test the architecture against the wording and intent distribution that arises in operational use; a larger evaluation set would also clarify whether the point-estimate correctness advantage of TLR over the chunk-based baseline stabilises or erodes with more data. Second, ingesting manufacturer technical manuals alongside the configuration data would supply the semantic context behind vendor mnemonics, supporting richer explanations and cross-manufacturer comparisons via retrieval-augmented generation over the manuals.

Acknowledgments This work has been funded by P&D CEMIG/ANEEL PD-04950-D0677/2023 supported by the Center of Excellence in Artificial Intelligence (CEIA), with financial resources from the MCTI grant number 057/2023, signed with EMBRAPII. This work was also supported by the National Institute of Science and Technology (INCT) in Responsible Artificial Intelligence for Computational Linguistics and Information Treatment and Dissemination (TILD-IAR) grant number 408490/2024-1.

Disclosure of Interests The authors declare no competing interests.

References

1. Biswal, A., Patel, L., Jha, S., Kamsetty, A., Liu, S., Gonzalez, J.E., Guestrin, C., Zaharia, M.: Text2SQL is not enough: Unifying AI and databases with TAG (2024). https://doi.org/10.48550/arXiv.2408.14717
2. Burdick, D., Danilevsky, M., Evfimievski, A.V., Katsis, Y., Wang, N.: Table extraction and understanding for scientific and enterprise applications. Proceedings of the VLDB Endowment **13**(12), 3433–3436 (2020). https://doi.org/10.14778/3415478.3415563
3. Granado, F., Lotufo, R., Pereira, J.: RAISE: Reasoning agent for interactive SQL exploration (2025). https://doi.org/10.48550/arXiv.2506.01273
4. Herzig, J., Müller, T., Krichene, S., Eisenschlos, J.M.: Open domain question answering over tables via dense retrieval. In: Proceedings of the Conference of the North American Chapter of the Association for Computational Linguistics (NAACL). pp. 512–519 (2021). https://doi.org/10.18653/v1/2021.naacl-main.43
5. Landis, J.R., Koch, G.G.: The measurement of observer agreement for categorical data. Biometrics **33**(1), 159–174 (1977). https://doi.org/10.2307/2529310
6. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.t., Rocktäschel, T., et al.: Retrieval-augmented generation for knowledge-intensive NLP tasks. In: Advances in Neural Information Processing Systems (NeurIPS) (2020). https://doi.org/10.48550/arXiv.2005.11401
7. Li, J., Hui, B., Qu, G., Yang, J., Li, B., Li, B., Wang, B., Qin, B., Geng, R., Huo, N., et al.: Can LLM already serve as a database interface? A BIg bench for large-scale database grounded text-to-SQLs. In: Advances in Neural Information Processing Systems (NeurIPS) (2023). https://doi.org/10.52202/075280-1835

8. Liu, Y., Iter, D., Xu, Y., Wang, S., Xu, R., Zhu, C.: G-Eval: NLG evaluation using GPT-4 with better human alignment. In: Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP). pp. 2511–2522 (2023). https://doi.org/10.18653/v1/2023.emnlp-main.153
9. Panickssery, A., Bowman, S.R., Feng, S.: LLM evaluators recognize and favor their own generations. In: Advances in Neural Information Processing Systems (NeurIPS) (2024). https://doi.org/10.52202/079017-2197
10. Pourreza, M., Rafiei, D.: DIN-SQL: Decomposed in-context learning of text-to-SQL with self-correction. In: Advances in Neural Information Processing Systems (NeurIPS) (2023). https://doi.org/10.52202/075280-1577
11. Scholak, T., Schucher, N., Bahdanau, D.: PICARD: Parsing incrementally for constrained auto-regressive decoding from language models. In: Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP). pp. 9895–9901 (2021). https://doi.org/10.18653/v1/2021.emnlp-main.779
12. de Souza, E.A., da Silva, P.F., Gomes, D., Batista, V., Batista, E., Pacheco, M.: TableRAG: A novel approach for augmenting LLMs with information from retrieved tables. In: Anais do XV Simpósio Brasileiro de Tecnologia da Informação e da Linguagem Humana (STIL). pp. 182–191 (2024). https://doi.org/10.5753/stil.2024.245371
13. Wang, B., Ren, C., Yang, J., Liang, X., Bai, J., Chai, L., Yan, Z., Zhang, Q.W., Yin, D., Sun, X., Li, Z.: MAC-SQL: A multi-agent collaborative framework for text-to-SQL. In: Proceedings of the 31st International Conference on Computational Linguistics (COLING). pp. 540–557 (2025). https://doi.org/10.48550/arXiv.2312.11242
14. Wang, K., Song, K., Xue, X., Zhao, Y.: Research on the method of constructing a knowledge map of relay protection device faults oriented to textual features. Scientific Reports **15**, 35063 (2025). https://doi.org/10.1038/s41598-025-08114-y
15. Wu, H., Shen, C., He, Z., Wang, Y., Xu, X.: SCADA-NLI: A natural language query and control interface for distributed systems. IEEE Access **9**, 78108–78127 (2021). https://doi.org/10.1109/ACCESS.2021.3083540
16. Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., Cao, Y.: ReAct: Synergizing reasoning and acting in language models. In: Proceedings of the International Conference on Learning Representations (ICLR) (2023). https://doi.org/10.48550/arXiv.2210.03629
17. Yu, T., Zhang, R., Yang, K., Yasunaga, M., Wang, D., Li, Z., Ma, J., Li, I., Yao, Q., Roman, S., Zhang, Z., Radev, D.: Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task. In: Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP). pp. 3911–3921 (2018). https://doi.org/10.18653/v1/D18-1425
18. Zheng, L., Chiang, W.L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E., et al.: Judging LLM-as-a-judge with MT-Bench and chatbot arena. In: Advances in Neural Information Processing Systems (NeurIPS) (2023). https://doi.org/10.52202/075280-2020
19. Zhu, F., Lei, W., Huang, Y., Wang, C., Zhang, S., Lv, J., Feng, F., Chua, T.S.: TAT-QA: A question answering benchmark on a hybrid of tabular and textual content in finance. In: Proceedings of the Annual Meeting of the Association for Computational Linguistics (ACL). pp. 3277–3287 (2021). https://doi.org/10.18653/v1/2021.acl-long.254